

# Úvod do jazyka Python

## Aritmetické výrazy

Vladimír Župka 2021

# Obsah

|                                                        |    |
|--------------------------------------------------------|----|
| Obsah .....                                            | 2  |
| Zpracování aritmetických výrazů .....                  | 3  |
| Kontrola aritmetického výrazu .....                    | 3  |
| Příklady správného výrazu .....                        | 3  |
| Neformální popis výrazu .....                          | 4  |
| Formální popis výrazu .....                            | 4  |
| Formální definice čísla .....                          | 4  |
| Hrubý popis činnosti stavového stroje .....            | 5  |
| Podrobný popis stavového stroje .....                  | 5  |
| Programové řešení .....                                | 6  |
| Hlavní funkce kontrola_aritmetickeho_vyrazu() .....    | 6  |
| Modul spolecne_funkce.py .....                         | 8  |
| globální data modulu: .....                            | 8  |
| funkce modulu: .....                                   | 8  |
| Zkouška kontroly výrazu .....                          | 15 |
| Výpočet aritmetického výrazu .....                     | 17 |
| Výraz bez závorek .....                                | 18 |
| Výraz se závorkami .....                               | 19 |
| Obecný postup výpočtu .....                            | 20 |
| Postup výpočtu se seznamy operandů a operátorů .....   | 21 |
| Unární operátory .....                                 | 22 |
| Hrubý popis stavového stroje .....                     | 22 |
| Příklad s unárními operátory .....                     | 24 |
| Podrobný popis stavového stroje .....                  | 25 |
| Programové řešení .....                                | 26 |
| Globální příkazy pro data .....                        | 26 |
| Hlavní výpočetní funkce .....                          | 26 |
| Pomocné vnitřní funkce .....                           | 28 |
| Příklady výpočtu aritmetického výrazu .....            | 29 |
| Výrazy se jmény proměnných a přiřazovací příkazy ..... | 30 |
| Podrobný popis stavového stroje .....                  | 30 |
| Moduly pro výpočet .....                               | 31 |
| Slovník .....                                          | 31 |
| Funkce prirazovaci_prikaz() .....                      | 31 |
| Příklady .....                                         | 33 |

# Zpracování aritmetických výrazů

V tomto textu budeme nadále používat *stavový stroj* k dalšímu využití ve zpracování aritmetických výrazů se závorkami a jmény proměnných. Využijeme k tomu předchozích poznatků o kontrole čísel.

- Nejprve budeme kontrolovat, zda je zápis výrazu správný a oznamovat chyby.
- Další úlohou bude vypočítat hodnotu aritmetického výrazu s číselnými operandy bez jmen proměnných.
- Nakonec budeme řešit výrazy s čísly i proměnnými. K tomu musíme vyřešit také přiřazovací příkaz, kterým definujeme hodnotu proměnné.

## Kontrola aritmetického výrazu

Úkolem bude kontrolovat správnost zápisu aritmetického výrazu zapsaného v textu. Ve výrazu mohou být použita čísla a jména jako *operandy* spojené s aritmetickými *operátory* pro sčítání, odčítání, násobení a dělení. Výraz může být uzavřen v *závorkách* a také *vnořen* do jiného (vnějšího) výrazu uzavřením do závorek.

## Příklady správného výrazu

123

$-1.45 + \text{pole\_12} * (a + b / (.2 - c)) + 62.$

$a + (1.45 + \text{pole\_28} * (a + b / (-2)) + (62. - - - 1)) - b$

$(1.45 + \text{pole1}) * (a + b / + -- 2 + 62. - b)$

Poznámka: Všimněme si, že znaménka  $+$  a  $-$  se mohou opakovat vícekrát za operátorem a před operandem. Chápou se jako *unární operátory* (případně vícenásobné) pro následující operand.

## Neformální popis výrazu

Předpis pro aritmetický výraz je *rekurentní*. To znamená, že je výraz je definován pomocí sebe sama. Takový předpis nelze vyjádřit regulárním výrazem, proto je vyjádřen rozšířenou Backus-Naurovovou formou (BNF).

Vzhledem k tomu, že i prostý operand s případným znaménkem je výraz, jsou možné kombinace výrazů bez závorek a výrazů se závorkami. Uzávorkované výrazy mohou být souřadné i podřadné (vnořené), jak ukazují výše uvedené příklady. Z naší definice vyplývá, že výraz nesmí být prázdný nebo složený se samých bílých znaků, nebo z těsné dvojice závorek.

## Formální popis výrazu

```
identifikátor ::= začátek pokračování
začátek ::= písmeno+
pokračování ::= [písmeno | číslice]*
písmeno ::= 'a'...'z' | 'A'...'Z' | '_'

white-space ::= ' ' | '\n' | '\t' | '\r' | '\v' | '\f'
mz          ::= white-space*

operátor ::= '+' | '-' | '*' | '/'
unární-operátor ::= '+' | '-'
levá-závorka ::= '('
pravá-závorka ::= ')'

operand ::= číslo | identifikátor

výraz ::= (mz operand mz) |
          (mz unární-operátor* mz výraz mz) |
          (mz výraz mz operátor mz výraz mz) |
          (mz levá-závorka mz výraz mz pravá-závorka mz)
```

Poznámka 1: Zkratka `mz` znamená mezery.

Poznámka 2: Mezi písmena se počítá i podtržítka `'_'`.

Poznámka 3: *Číslem* budeme rozumět zlomkové číslo s rozdělovacími podtržítka a případným exponentem, tzv. vědecký tvar (např.  $1.234e5$ , tedy  $1.234 \times 10^5$ ).

### Formální definice čísla

```
cislo_se_znamenkem ::= ['+' | '-'] cislo
cislo                ::= celky | zlomkove_cislo | exponentove_cislo
zlomkove_cislo       ::= [celky] zlomek | celky '.'
exponentove_cislo    ::= (celky | zlomkove_cislo) exponent
celky                ::= číslice (['_'] číslice)*
zlomek               ::= '.' celky
exponent              ::= ('e' | 'E') ['+' | '-'] celky
číslice              ::= '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9'
```

Svislá čárka `|` znamená "nebo". Výraz v hranatých závorkách `[ ]` je nepovinný. Znaménko `+` za výrazem znamená, že výraz musí být přítomen aspoň jednou a může se libovolně opakovat. Hvězdička `*` za výrazem znamená, že výraz se může libovolně opakovat, ale nemusí být vůbec přítomen. Výraz v kulatých závorkách `( )` je povinná skupina. Některé značky se shodují se značkami používanými v regulárních výrazech (plus, hvězdička, svislá čárka, kulaté závorky), ale některé se používají jinak (zdejší hranaté závorky jsou v regulárním výrazu vyjádřeny otazníkem).

## Hrubý popis činnosti stavového stroje

Převod ze syntaktického předpisu do programu není přímočarý, je možná zapotřebí několika pokusů, než je stavový stroj správně zkonstruován. Zde bude uvedeno jedno řešení.

Zpracování textu probíhá v cyklu tak, že ze souboru nebo z řetězce se čte znak po znaku se třemi znaky dopředu. Program přitom bude rozeznávat následující stavy:

|                    |                                                             |
|--------------------|-------------------------------------------------------------|
| PRED_OPERANDEM     | na začátku, po přečtení operátoru, po přečtení levé závorky |
| ZA_OPERANDEM       | byl zpracován operand                                       |
| ZA_PRAVOU_ZAVORKOU | byla přečtena pravá závorka                                 |
| konec výrazu       |                                                             |

V každém stavu program vyhodnotí přečtený znak, přečte (nebo nepřečte) další znak (znaky), a přejde do jiného stavu nebo zůstane ve stejném stavu. Nepovolené znaky hlásí jako chybné.

## Podrobný popis stavového stroje

Začíná stavem PRED\_OPERANDEM a čte první znak. Pak opakuje cyklus, dokud nepřečte poslední znak. Když byl ve stavu

PRED\_OPERANDEM přečten znak

+ nebo –, přečte další znak a zůstane ve stejném stavu

**číslice** nebo **tečka**, provede funkci `cislo_exp()` a přejde do stavu ZA\_OPERANDEM

**písmeno**, provede funkci `identifikator()` a přejde do stavu ZA\_OPERANDEM

**levá závorka**, přičte jedničku k čítači levých závorek, přečte další znak a přejde do stavu PRED\_OPERANDEM (*vstoupí do rekurze*)

**mezera** (white-space), přečte další znak a zůstane ve stejném stavu

**jiný**, ohlásí chybu, přečte další znak a zůstane ve stejném stavu

ZA\_OPERANDEM přečten znak

**operátor**, přečte další znak a přejde do stavu PRED\_OPERANDEM

**pravá závorka**, přičte jedničku k čítači pravých závorek, přečte další znak a přejde do stavu ZA\_PRAVOU\_ZAVORKOU (*vystoupí z rekurze*)

**mezera** (white-space), přečte další znak a zůstane ve stejném stavu

**jiný**, ohlásí chybu, přečte další znak a zůstane ve stejném stavu

ZA\_PRAVOU\_ZAVORKOU přečten znak

**operátor**, přečte další znak a přejde do stavu PRED\_OPERANDEM

**pravá závorka**, přičte jedničku k čítači pravých závorek, přečte další znak a zůstane ve stejném stavu

**mezera** (white-space), přečte další znak a zůstane ve stejném stavu

**jiný**, ohlásí chybu, přečte další znak a zůstane ve stejném stavu

Když cyklus skončí ve stavu PRED\_OPERANDEM, ohlásí program chybu, že za operátorem chybí výraz, nebo chybu v počtu závorek.

# Programové řešení

Program je zapsán v *modulu* `arit_vyraz_kontrola.py`. Modul se skládá z *hlavní funkce* `kontrola_aritmetického_vyrazu()` a několika *pomocných funkcí*. Parametr `indata` je buď otevřený textový soubor, nebo textový řetězec. Hlavní funkce používá další funkce a třídy z modulu `spolecne_funkce.py`, který je použit jako předpona `sf.` před názvy funkcí.

## Hlavní funkce `kontrola_aritmetického_vyrazu()`

```
import spolecne_funkce as sf

def kontrola_aritmetického_vyrazu(indata):
    """
    *** Funkce kontroluje správnost aritmetického výrazu
    se závorkami a vnořováním.
    Parametr indata je buď otevřený vstupní soubor nebo string.
    """

    PRED_OPERANDEM = 1
    ZA_PRAVOU_ZAVORKOU = 2
    ZA_OPERANDEM = 3
    pocet_lz = 0
    pocet_pz = 0

    stav = PRED_OPERANDEM

    sf.prvni3(indata) # čtu první tři znaky
    if sf.konec:
        sf.chyba(f"prázdný vstup")

    # Začátek cyklu
    while True:
        if sf.konec:
            break

        if stav == PRED_OPERANDEM:
            # čekám operátor, číslici, písmeno, levou závorku
            if sf.znak == '+' or sf.znak == '-':
                print(sf.znak, end='')
                sf.dalsi3(indata)
                continue
            elif sf.je_cislice(sf.znak) or sf.znak == '.':
                cis = sf.cislo_exp(indata)
                print(cis, end='')
                stav = ZA_OPERANDEM
                continue
            elif sf.je_pismeno(sf.znak):
                sf.identifikator(indata)
                stav = ZA_OPERANDEM
                continue
            elif sf.znak == '(':
                print(sf.znak, end='')
                pocet_lz += 1
                stav = PRED_OPERANDEM
                sf.dalsi3(indata)
                continue
            elif sf.je_mezera(sf.znak):
                print(sf.znak, end='')
                sf.dalsi3(indata)
                continue
            else:
                print(sf.znak, end='')
                sf.chyba(f"chybný znak {sf.znak} ve stavu PRED_OPERANDEM")
                sf.dalsi3(indata)
                continue
```

```

if stav == ZA_OPERANDEM: # čekám operátor, pravou závorku
    if sf.je_operator(sf.znak):
        print(sf.znak, end='')
        stav = PRED_OPERANDEM
        sf.dalsi3(indata)
        continue
    elif sf.znak == ')':
        print(sf.znak, end='')
        pocet_pz += 1
        if pocet_pz > pocet_lz:
            sf.chyba(f"chybí levá závorka")
            stav = ZA_PRAVOU_ZAVORKOU
            sf.dalsi3(indata)
            continue
    elif sf.je_mezera(sf.znak):
        print(sf.znak, end='')
        sf.dalsi3(indata)
        continue
    else:
        print(sf.znak, end='')
        sf.chyba(f"chybný znak {sf.znak} ve stavu ZA_OPERANDEM")
        sf.dalsi3(indata)
        continue
if stav == ZA_PRAVOU_ZAVORKOU:
    if sf.je_operator(sf.znak):
        print(sf.znak, end='')
        stav = PRED_OPERANDEM
        sf.dalsi3(indata)
        continue
    elif sf.znak == ')':
        print(sf.znak, end='')
        pocet_pz += 1
        if pocet_pz > pocet_lz:
            sf.chyba(f"chybí levá závorka")
            sf.dalsi3(indata)
            continue
    elif sf.je_mezera(sf.znak):
        print(sf.znak, end='')
        sf.dalsi3(indata)
        continue
    else:
        print(sf.znak, end='')
        sf.chyba(f"chybný znak {sf.znak} ve stavu ZA_PRAVOU_ZAVORKOU")
        sf.dalsi3(indata)
        continue
# Konec cyklu
sf.konec = False # zruším příznak koce pro příští volání
if stav == PRED_OPERANDEM:
    sf.chyba(f"\nNa konci chybí výraz")
if pocet_lz != pocet_pz:
    sf.chyba(f"\npočet levých závorek: {pocet_lz}, "
            f"počet pravých závorek: {pocet_pz}")
if sf.nastala_chyba:
    print("Ve výrazu jsou chyby!")
    return False
else:
    print("\nVýraz je správný.")
    return True

```

## Modul `spolecne_funkce.py`

### globální data modulu:

```
ind = 0 # index současného znaku
znak, znak2, znak3 = '', '', '' # znaky čtené dopředu
nastala_chyba = False # příznak aspoň jedné chyby
konec = False # příznak konce vstupních dat
konec3 = False # pomocný příznak konce vstupních dat
slovník = dict() # tabulka jmén proměnných s jejich hodnotami
```

### funkce modulu:

```
cislo_exp(indata)
identifikator(indata)
dalsi3(indata)
je_cislice(character)
je_pismeno(character)
je_mezera(character)
je_operator(character)
chyba(string)
```

### třídy pro zásobník:

```
Polozka(list)
Zasobnik
```



### Funkce `cislo_exp()`

Tato funkce zkontroluje znak po znaku vstupní text, zda tvoří zlomkové číslo. První znak čísla ze vstupního textu (parametru `indata`) musí být znám již při vyvolání z hlavní funkce. Další znaky si funkce čte pomocí funkce `dalsi3()`. Přečte-li nepovolený znak, případně ohlásí neúplné číslo spočívající v osamělé tečce, vytiskne číslo a končí. Stavový stroj je podobný tomu, který byl použit v předchozí části kurzu ke kontrole textu s oddělovacími čárkami. Má jen čtyři stavy, protože kontroluje jen jedno číslo. Ke kontrole podtržítka mezi třemi znaky používá pomocnou funkci volanou třikrát, ve stavech `CELKY`, `ZLOMEK`, `CIS_EXP`.

```
def cislo_exp(indata):
    global konec
    exp_cislo = '' # řetězec pro strádání znaků exp. čísla
    CELKY = 1
    ZLOMEK = 2
    EXP = 3
    CIS_EXP = 4

    def kontrola_podtrzitka():
        nonlocal exp_cislo, indata
        if je_cislice(znak):
            exp_cislo += znak
            if znak2 == '_': # druhý znak je podtržítko?
                if je_cislice(znak3): # a třetí znak je číslice?
                    pass # podtržítko znak2 NEpřijde do čísla
                else:
                    chyba(f"chybný {ind+1}. znak {repr(znak2)} "
                          f"v čísle za znakem {repr(znak)}")
                    # přeskočím znak podtržítko (čili další znak přečtu dvakrát)
                    dalsi3(indata)
                    dalsi3(indata)
                    return
            dalsi3(indata)
            return
        elif znak == '_' and not je_cislice(znak2):
            # první je podtržítko a druhý není číslice?
            chyba(f"chybný {ind+1}. znak {repr(znak)} "
                  f"v čísle před {repr(znak2 + znak3)}")
            dalsi3(indata)
            return
        elif znak == '_' and je_cislice(znak2):
            # první je podtržítko a druhý je číslice?
            dalsi3(indata) # přeskočím ho
            return

    def zpracovani():
        nonlocal exp_cislo
        if not any(c.isdigit() for c in exp_cislo):
            chyba(f"neúplné číslo: {repr(exp_cislo)}")

    stav = CELKY

    while True: # první znak je už přečten a je to číslice nebo tečka
        if konec: # prázdný znak značí konec souboru
            break # už ho nezpracuji, aby se na něm nehlásila chyba
        if stav == CELKY:
            if je_cislice(znak):
                kontrola_podtrzitka()
                continue
            if znak == '.': # tečka?
                exp_cislo += znak
```

```

        stav = ZLOMEK
        dalsi3(indata)
        continue
    elif znak == 'e' or znak == 'E': # exponent?
        exp_cislo += znak
        stav = EXP
        dalsi3(indata)
        continue
    else: # končím číslo v celku, protože nepřišla značka E nebo e
        break
elif stav == ZLOMEK:
    if je_cislice(znak):
        kontrola_podtržitka()
        continue
    elif znak == '_' and je_cislice(znak2):
        # první je podtržítka a druhý je číslice?
        dalsi3(indata) # přeskočím ho
        continue
    elif znak == 'e' or znak == 'E': # exponent?
        exp_cislo += znak
        stav = EXP
        dalsi3(indata)
        continue
    else: # končím číslo ve zlomku, protože nepřišla značka E nebo e
        break
elif stav == EXP:
    if znak == '+' or znak == '-':
        exp_cislo += znak
        stav = CIS_EXP
        dalsi3(indata)
        continue
    elif je_cislice(znak):
        stav = CIS_EXP
        continue # nečtu další a předávám do čísla
    else:
        chyba(f"          chybný {ind}. znak {repr(znak)} "
              f"v čísle před {repr(znak2 + znak3)}")
        dalsi3(indata)
        continue
elif stav == CIS_EXP:
    if je_cislice(znak):
        kontrola_podtržitka()
        continue
    else:
        break # když znak není číslice, končím číslo
# Konec cyklu
zpracovani() # zpracuji poslední číslo
return exp_cislo

```

### Funkce `identifikator()`

Tato funkce zkontroluje znak po znaku vstupní text, zda tvoří identifikátor (jméno v Pythonu). První znak identifikátoru (písmeno) ze vstupního textu (parametru `indata`) musí být znám již při vyvolání. Další znaky si funkce čte pomocí funkce `dalsi3()`. Přečte-li nepovolený znak, končí a vrátí nastřádaný řetězec jako identifikátor.

```
def identifikator(indata):
    global znak
    ZACATEK = 1
    POKRACOVANI = 2
    ident = '' # řetězec pro střezení znaků identifikátoru

    stav = ZACATEK

    while True:
        if stav == ZACATEK:
            if je_pismeno(znak):
                ident += znak
                stav = POKRACOVANI # pokračuji za prvním písmenem
                dalsi3(indata)
            else:
                break # nepovolený znak – končím
        if stav == POKRACOVANI:
            if je_pismeno(znak) or je_cislice(znak):
                ident += znak
                dalsi3(indata)
                continue
            else:
                break # nepovolený znak – končím
    return ident
```

### Funkce `prvni3()`

Tato funkce přečte ze vstupu `indata` první tři znaky, jsou-li k dispozici. Není-li ve vstupu ani první znak, nastaví globální příznak `konec = True`. Druh vstupu rozlišuje podle toho, zda parametr `indata` obsahuje atribut "`read`", což znamená, že jde o otevřený vstupní soubor. Když tento atribut v objektu chybí, rozhodne, že jde o prostý textový řetězec (objekt typu `str`). V souboru čte znak metodou `read()`, v řetězci čte znak podle indexu.

```
def prvni3(indata):
    global znak, znak2, znak3, konec, ind
    if hasattr(indata, "read"):
        # vstup je otevřený vstupní soubor
        znak = indata.read(1)
        if znak == '':
            konec = True
            return
        znak2 = indata.read(1)
        znak3 = indata.read(1)
    else: # vstup je text
        if ind == len(indata):
            konec = True
            return
        else:
            znak = indata[ind]
            if ind + 1 < len(indata):
                znak2 = indata[ind + 1]
            if ind + 2 < len(indata):
                znak3 = indata[ind + 2]
            ind += 1 # posunu index znaku o 1 místo
```

### Funkce `dalsi3()`

Tato funkce čte další tři znaky v pořadí ze vstupního textu (parametru `indata`), jsou-li k dispozici. Druh vstupu rozlišuje stejně jako funkce `prvni3()`. Není-li ve vstupu ani jeden další znak, nastaví globální příznak `konec = True`.

```
def dalsi3(indata):
    global znak, znak2, znak3, konec, ind
    if hasattr(indata, "read"):
        # vstup je otevřený vstupní soubor
        znak = znak2
        if znak == '':
            konec = True
            return
        znak2 = znak3
        znak3 = indata.read(1)
    else: # vstup je text
        if ind < len(indata):
            znak = indata[ind]
        else:
            konec = True
            znak = '' # smažu znak přečtený na konci, aby se netiskl
            return
        if ind + 1 < len(indata):
            znak2 = indata[ind + 1]
        if ind + 2 < len(indata):
            znak3 = indata[ind + 2]
        ind += 1 # posunu index znaku o 1 místo
```

### Další pomocné funkce

Tyto funkce jsou velmi jednoduché a nevyžadují zvláštní vysvětlení.

```
def je_cislice(character): # je číslice?
    if character in "0123456789":
        return True
    else:
        return False

def je_pismeno(character):
    if character in "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ_":
        return True
    else:
        return False

def je_mezera(character): # white-space
    if character in " \n\t\r\v\f":
        return True
    else:
        return False

def je_operator(character): # operátor
    if character in "+-*/":
        return True
    else:
        return False

def chyba(string): # nastavení příznaku a hlášení chyby
    global nastala_chyba
    nastala_chyba = True
    print(f"\n          {string}")
```

## Třídy pro zásobník

jsou stejné jako v publikaci o třídách.

```
# Položka zásobníku
class Polozka():
    # Vytvoří novou položku
    def __init__(self, data):
        self.data = data
        self.dalsi = None # ukazatel na další položku

# Zásobník položek
class Zasobnik:
    # Vytvoří nový zásobník s prázdným začátkem.
    def __init__(self):
        self.top = None # vršek zásobníku

    # Vráti True, je-li zásobník prázdný, jinak vrátí False.
    def empty(self):
        return self.top is None

    # Odstraní položku ze začátku zásobníku a vrátí její data.
    def pop(self):
        if self.empty():
            raise MemoryError(f"Zásobník {self} je prázdný!")
        stara_top = self.top # schová první položku
        self.top = stara_top.dalsi # zamění odebíranou položku za další
        return stara_top.data # vrátí data odebrané položky

    # Vtlačí (přidá) novou položku na top zásobníku.
    def push(self, data):
        nova_polozka = Polozka(data) # nová položka
        nova_polozka.dalsi = self.top # doplním ukazatel na další položku
        self.top = nova_polozka # další položka se stane začáteční
```

Následuje popis funkcí modulu.

## Zkouška kontroly výrazu

Společné funkce jsou zapsány v modulu `spolecne_funkce.py`. Modul importujeme pod jménem `sf`.

```
import spolecne_funkce as sf
```

```
print("*Test výrazu:")
```

```
text = "-1.45 + _pole_12 * (a + b / (2-c) ) + 62."
kontrola_aritmetického_vyrazu(text)
```

```
-1.45 + _pole_12 * (a + b / (2-c) ) + 62.
Výraz je správný.
```

```
text = "1.45+_pole_12*(a+b/(.2-c))+62."
kontrola_aritmetického_vyrazu(text)
```

```
1.45+_pole_12*(a+b/(.2-c))+62.
Výraz je správný.
```

```
text = "a + (1.45 + pole_28 * (a + b /(-2) ) + (62. - + 1)) - b"
kontrola_aritmetického_vyrazu(text)
```

```
a + (1.45 + pole_28 * (a + b /(-2) ) + (62. - + 1)) - b
Výraz je správný.
```

```
text = "(1.45 + pole1) * (a + b /+--2 + 62. - b)"
kontrola_aritmetického_vyrazu(text)
```

```
(1.45 + pole1) * (a + b /+--2 + 62. - b)
Výraz je správný.
```

```
with open("cisla.txt", "w") as outfile: # zápis zkušebních dat do souboru
    outfile.write("2*3*(1+(3---+1)+1-(1/6))")
infile = open("cisla.txt", "r") # vstup je otevřený soubor
kontrola_aritmetického_vyrazu(infile)
```

```
2*3*(1+(3---+1)+1-(1/6))
Výraz je správný.
```

```

print("*Chybné výrazy:")

text = "2.5 + * 3"
kontrola_aritmetického_vyrazu(text)
2.5 + *          chybný znak * ve stavu PRED_OPERANDEM
3
Ve výrazu jsou chyby!

text = "2.5 + (3 * (b+c)"
kontrola_aritmetického_vyrazu(text)
2.5 + (3 * (b+c)    počet levých závorek: 2, počet pravých závorek: 1

Ve výrazu jsou chyby!

text = "a 2.5 + 3 / (b)"
kontrola_aritmetického_vyrazu(text)
a 2          chybný znak 2 ve stavu ZA_OPERANDEM
.            chybný znak . ve stavu ZA_OPERANDEM
5            chybný znak 5 ve stavu ZA_OPERANDEM
+ 3 / (b)
Ve výrazu jsou chyby!

text = "@ 2.5 + -3 *"
kontrola_aritmetického_vyrazu(text)
@            chybný znak @ ve stavu PRED_OPERANDEM
2.5 + -3 *    na konci chybí výraz

Ve výrazu jsou chyby!

```



# Výpočet aritmetického výrazu

Formální popis aritmetického výrazu je stejný jako v kapitole o kontrole aritmetického výrazu viz [formální popis výrazu](#). Jeho formální správnost už dokážeme zkontrolovat. Vypočítat jeho hodnotu je ale mnohem složitější práce, i když jen s číselnými hodnotami bez jmen proměnných. Vyžaduje to *okamžitý převod* řetězců do čísel (`float` nebo `Decimal`), provádění operací, ukládání mezivýsledků do *zásobníku* a jejich vybírání ze zásobníku při rekurzi, a to vše značně komplikuje program.

Pro výpočet aritmetického výrazu se jmény proměnných je ještě nutné zavést možnost přidat jménům číselnou hodnotu pomocí přiřazovacího příkazu obsahujícího na pravé straně výraz s číselnými operandy a ukončující středník, například

```
a = 2;  
b = 3 ;  
c = 1.6 * (2.4 - .4) ;
```

Přiřazovací příkaz pak můžeme rozšířit i na případ obsahující na pravé straně také *jména* (*identifikátory*) definované dříve:

```
vysledek = 0.23 + (a + b) * c ;
```

Při výpočtu pak do výrazu dosadíme hodnoty místo proměnných:

```
vysledek = 0.23 + (2 + 3) * 3.2 = 16.23 ;
```

Pak můžeme i proměnnou `vysledek` použít v některém dalším výrazu, protože získala číselnou hodnotu. *Středník* použijeme k ukončení výrazu. Tím se z výrazu stane příkaz.

Nejprve si probereme jednodušší úlohu, a to výpočet aritmetického výrazu obsahujícího vnořené závorky a *pouze číselné operandy* i s *unárními operátory* `+` nebo `-`. Unární operátor `-` (minus) mění znaménko následujícího operandu, kdežto unární operátor `+` je nemění. Pro zjednodušení výkladu budeme předpokládat, že výraz je formálně správný. Při kontrole správnosti výrazu nezáleželo na pořadí operátorů, ale při výpočtu na něm záleží. Platí totiž *přednost násobení a dělení* před sčítáním a odčítáním, nejsou-li přitom použity závorky.

## Výraz bez závorek

Začneme výrazem *bez závorek a bez unárnych operátorů*. Za první příklad zvolíme výraz

$$1 + 1 + 2 * 3$$

Při postupu zleva nemůžeme hned sčítat operandy  $1 + 1 + 2$ , protože takto získané číslo 4 později násobené číslem 3 by dalo výsledek 12. To by odporovalo pravidlu o přednosti, které říká že *násobení a dělení má přednost* před sčítáním a odčítáním. Proto musíme nejprve spočítat součin  $2 * 3 = 6$ , pak sečíst  $1 + 1 + 6$  a dostaneme správný výsledek 8.

Z toho můžeme odvodit *pravidlo*, že z výrazu musíme nejprve vypočítat všechny operace násobení a dělení. Jejich výsledky dosadíme do míst jejich levých operandů, přičemž pravé operandy i s operátory smažeme. Potom teprve provést všechny operace sčítání a odčítání, a to *zleva doprava*, kvůli odčítání, které je *asociativní zleva* (operandy odčítání se po dvou sdružují zleva).

Kdybychom postupovali zprava doleva, dostali bychom nesprávný výsledek. Například výraz  $4 - 2 - 2$  vyhodnocený zleva má hodnotu 0, stejnou jako výraz  $(4 - 2) - 2$ , ale stejný výraz vyhodnocený zprava má hodnotu 4, stejnou jako výraz  $4 - (2 - 2)$ .

*Rozdíl vyhodnocování výrazu zleva a zprava*

Vyhodnocení výrazu zleva (správné)

$$\begin{aligned} & \underline{1+2*2*2-1-2*3+1} \\ & 1+8-1-6+1 \\ & 9-1-6+1 \\ & 8-6+1 \\ & 2+1 \\ & 3 \end{aligned}$$

Vyhodnocení výrazu zprava (nesprávné)

$$\begin{aligned} & \underline{1+2*2*2-1-2*3+1} \\ & 1+8-1-6+1 \\ & 1+8-1-7 \\ & 1+8--6 \\ & 1+14 \\ & 15 \end{aligned}$$

## Výraz se závorkami

Vyložíme postup výpočtu s ohledem na závorky a přednost operátorů, *zatím bez unárních operátorů*.

Při výpočtu postupujeme zleva doprava tak, že čteme znaky, vynecháváme mezery, a čísla převádíme ze znakové do číselné podoby.

Jakmile narazíme na *levou závorku*, musíme *nedořešenou část* výrazu uložit *do zásobníku* a zahájit nový výraz vstupem do rekurze. Když v něm zase narazíme na *levou závorku*, další nedořešenou část podvýrazu uložíme *do zásobníku* atd. Jakmile narazíme na *pravou závorku*, vystupujeme z rekurze, vypočítáme hodnotu nezávorkového výrazu; výsledek bez závorek přidáme k nedořešené části, kterou jsme vybrali *ze zásobníku* atd. Na konci celého výrazu vypočítáme zbývající nezávorkový výraz.

Jako příklad si uvedeme výraz

$4.5 + 1 + ( 7 - ( 10 - 2 * 4 ) ) / 5$

Můžeme si jej schematicky znázornit takto:

nedořešená-část ( nedořešená-část ( nezávorkový-výraz ) ) další-část

$4.5 + 1 + ( 7 - ( 10 - 2 * 4 ) ) / 5$

Postupné řešení tohoto výrazu dává následující mezivýsledky.

$4.5 + 1 + ( 7 - ( 10 - 2 * 4 ) ) / 5$

$4.5 + 1 + ( 7 - 2 ) / 5$

$4.5 + 1 + 5 / 5$  *zbývající nezávorkový výraz*

$6.5$

## Obecný postup výpočtu

$$4.5 + 1 + ( 7 - ( 10 - 2 * 4 ) ) / 5$$

| Zásobník          | Průběžné výsledky řešení výrazu        |                  |
|-------------------|----------------------------------------|------------------|
| prázdný           | $4.5 + 1 + ( 7 - ( 10 - 2 * 4 ) ) / 5$ |                  |
| [4.5 + 1 +]       | $( 7 - ( 10 - 2 * 4 ) ) / 5$           | vstup do rekurze |
| [4.5 + 1 +] [7 -] | $( 10 - 2 * 4 ) / 5$                   | vstup do rekurze |
| [4.5 + 1 +] [7 -] | $10 - 8 ) ) / 5$                       |                  |
| [4.5 + 1 +] [7 -] | $2 ) ) / 5$                            |                  |
| [4.5 + 1 +]       | $7 - 2 ) / 5$                          | výstup z rekurze |
| prázdný           | $4.5 + 1 + 5 / 5$                      | výstup z rekurze |
| prázdný           | $4.5 + 1 + 1$                          |                  |
| prázdný           | $5.5 + 1$                              |                  |
| prázdný           | $6.5$                                  |                  |

- Čteme znaky zleva a jakmile narazíme na první *levou* závorku ( , uložíme nedořešenou část  $4.5 + 1 +$  do zásobníku a *vstupujeme do rekurze*.
- Čteme další znaky a zase, když přečteme druhou *levou* závorku ( , uložíme nedořešenou část  $7 -$  do zásobníku a opět *vstupujeme do rekurze*.
- Čteme další část, dokud nenarazíme na první *pravou* závorku ) . V té chvíli víme, že mezi touto závorkou a poslední levou závorkou leží nezávorkový výraz  $10 - 2 * 4$ . Ten vyřešíme podle našeho pravidla tak, že nejprve vyřešíme výraz  $2 * 4$  s výsledkem 8. Pak ten výraz smažeme a na jeho místo dosadíme výsledek 8.
- Dostáváme výraz  $10 - 8$ , který vyřešíme s výsledkem 2. Výraz smažeme a na jeho místo dosadíme výsledek 2. Čteme další znak, což je první *pravá* závorka ) .
- *Výstupujeme z rekurze* tak, že vyjmeme ze zásobníku naposledy uloženou nedořešenou část  $7 -$ , a k ní přidáme výsledek nezávorkového výrazu – číslo 2.
- Dostáváme výraz  $7-2$  a vyřešíme nezávorkový výraz s výsledkem 5. Pak ten výraz smažeme a výsledek 5 dosadíme na jeho místo. Čteme další znak, což je druhá *pravá* závorka ) .
- *Výstupujeme z rekurze* tak, že vyjmeme ze zásobníku poslední nedořešenou část  $4.5 + 1 +$  a k ní přidáme výsledek nezávorkového výrazu – číslo 5. Dostáváme  $4.5 + 1 + 5$  a čteme další část výrazu  $/ 5$ , kterou přidáme na konec. Dostáváme  $4.5 + 1 + 5 / 5$ .
- Dalším čtením zjistíme konec textu, což znamená, že máme k dispozici poslední nezávorkový výraz, který vyřešíme podle našeho pravidla a dostáváme postupně  $4.5 + 1 + 1$ ,  $5.5 + 1$  a **6.5**, což je konečný výsledek výrazu.

## Postup výpočtu se seznamy operandů a operátorů

Při řešení výpočtu je výhodné ukládat průběžné výsledky do dvou *seznamů*, jednoho pro operandy a druhého pro operátory. Seznamy z nedořešených částí budeme ukládat do *zásobníků*, také jednoho pro operandy a druhého pro operátory.

$4.5 + 1 + ( 7 - ( 10 - 2 * 4 ) ) / 5$

- Přečteme znaky **4.5 + 1 +** před závorkou, přičemž "číselné" textové řetězce proměníme na čísla a zapisujeme je do seznamu operandů pro pozdější použití. Operátory zapisujeme do seznamu operátorů. Seznam operandů bude [ 4.5, 1 ] a seznam operátorů bude [ +, + ].
- Když přečteme *levou závorku*, seznamy uložíme do *zásobníků* pro použití až v době, kdy vyhodnotíme obsah mezi touto levou závorkou a odpovídající pravou závorkou.
- Přečteme číslo 7 a operátor **−** (minus) a oba znaky uložíme do seznamů [ 7 ] a [ − ].
- Přečteme *další levou závorku* a seznamy [ 7 ] a [ − ] uložíme do zásobníků.
- Čteme znaky **10 − 2 \* 4** a zapisujeme čísla a operátory do seznamů [ 10, 2, 4 ] a [ −, \* ].
- Přečteme *první pravou závorku*. Nyní, když víme, že mezi poslední levou závorkou a touto pravou závorkou již žádné závorky nejsou, můžeme vyhodnotit obsah posledních seznamů. Postupujeme podle našeho pravidla o nezávorkových výrazech.
- V seznamech [ 10, 2, 4 ] a [ −, \* ] přeskočíme operátor **−** (minus) a jeho levý operand 10, vyhodnotíme operátor **\*** a jeho operandy 2 a 4 s výsledkem 8 a pak operátor i pravý operand smažeme a výsledek dosadíme na místo levého operandu. Z toho vzniknou seznamy [ 10, 8 ] a [ − ]. Provedeme operaci odečtení  $10 - 8$ , smažeme operátor **−** a pravý operand 8. Výsledek 2 dosadíme na místo levého operandu 10. Získáme tak seznamy [ 2 ] a [ ].
- Nyní máme vyřešený obsah *vnitřních* závorek, a tak přečteme další znak **−** *druhou pravou závorku*.
- Teď vybereme ze zásobníků seznamy [ 7 ] a [ − ] a zařadíme výslednou hodnotu 2 na konec seznamu operandů. Dostáváme: [ 7, 2 ] a [ − ]. Seznam operátorů byl prázdný, proto do něj žádný operátor nepřidáváme. Teď můžeme provést odečtení  $7 - 2$  a získáme seznamy [ 5 ] a [ ].
- Nyní máme vyřešený obsah *vnějších* závorek. Vybereme ze zásobníků seznamy [ 4.5, 1 ] a [ +, + ], čteme další znak **/** (operátor dělení) a číslo 5. K vybraným seznamům přidáme poslední výsledek 5 a přečtené znaky **/ a 5**: [ 4.5, 1, 5, 5 ] a [ +, +, / ].
- Postupujeme opět podle našeho pravidla o nezávorkových výrazech a dostaneme po dělení  $5/5$  postupně seznamy [ 4.5, 1, 1 ] a [ +, + ], [ 5.5, 1 ] a [ + ]. a nakonec [ 6.5 ] a [ ].
- Po zjištění konce textu dostáváme celkový výsledek výrazu číslo **6.5**.

## Unární operátory

V dosavadních příkladech jsme měli v operandech čísla bez znaménka, například 4.5, 1, 7 atd. Znaménko + nebo – před číslem, například –1.1, +1.1 se v Pythonu považuje za *unární operátor*. Chceme-li mít v aritmetickém výrazu číselné operandy se znaménky, tedy s unárními operátory, musíme je rozlišit od *binárních operátorů* +, – umístěných mezi dvěma operandy.

## Hrubý popis stavového stroje

Do tohoto popisu je zařazeno kromě řešení unárních operátorů také vyhodnocování jmen proměnných (identifikátorů) s předpokladem, že do proměnných byly předem dosazeny hodnoty pomocí přiřazovacích příkazů. Přiřazovací příkazy probereme později.

Rozeznáme stavy

|                 |                                                                                                                   |
|-----------------|-------------------------------------------------------------------------------------------------------------------|
| PRED_OPERANDEM  | na začátku, po přečtení operátoru (+ – * /) ve stavu<br>ZA_OPERANDEM, po přečtení levé závorky (vstup do rekurze) |
| ZA_OPERANDEM    | po zjištění čísla (přečtení čísllice nebo tečky), po zjištění<br>identifikátoru (přečtení písmena)                |
| UNARNI_OPERATOR | po přečtení + nebo – ve stavu PRED_OPERANDEM                                                                      |
| konec výrazu    |                                                                                                                   |

### Ve stavu PRED\_OPERANDEM

Při zjištění *čísla* (čísllice nebo tečky) se vyvolá funkce pro kontrolu čísla `cislo_exp()`, které se předá přečtený znak. Funkce pak sama čte další znaky a končí přečtením dalšího znaku za zpracovaným číslem. Proto se ve stavovém stroji už nečte další znak, ale pokračuje se dále. Číslo se přidá k seznamu operandů a provedou se přednostní operace násobení a dělení ze seznamu. Přejde se do stavu ZA\_OPERANDEM.

Při zjištění *identifikátoru* (písmena) se vyvolá funkce `identifikator()`, která přečte celé jméno proměnné a končí přečtením dalšího znaku za zpracovaným jménem. Proto se ve stavovém stroji už nečte další znak, ale pokračuje se dále. Podle jména se vyhledá ve slovníku jeho hodnota a přidá se k seznamu operandů. Přejde se do stavu ZA\_OPERANDEM.

Při zjištění *unárního operátoru* (znaménko + nebo – ve stavu PRED\_OPERANDEM) se k seznamu operandů přidá číslo +1 nebo –1 a přejde se do stavu UNARNI\_OPERATOR.

Při zjištění *levé závorky* (vstup do rekurze) se uloží seznamy operandů a operátorů do zásobníku a přejde se do stavu PRED\_OPERANDEM.

### Ve stavu ZA\_OPERANDEM

Při zjištění operátoru (+ – \* /) se přidá operátor do seznamu a přejde se do stavu PRED\_OPERANDEM.

Při zjištění *pravé závorky* (výstup z rekurze) se provedou zbývající operace ze seznamu, vyberou se seznamy ze zásobníku, přidá se k nim výsledek zbývajících operací a provede se násobení a dělení ze seznamu.

### Ve stavu UNARNI\_OPERATOR

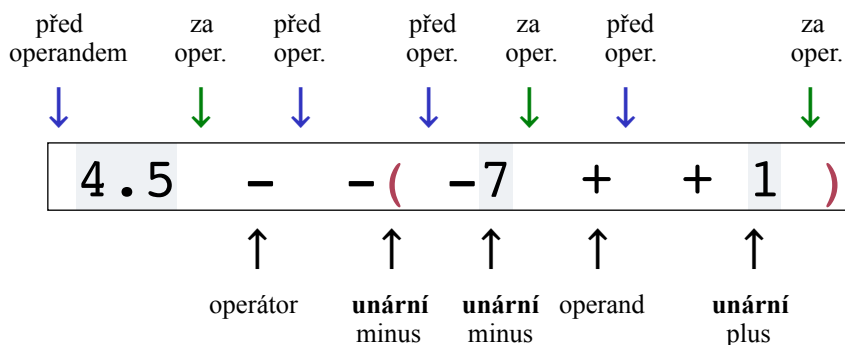
Když se zjistí znaménko minus, obrátí se znaménko posledního operandu seznamu.

Když se zjistí *číslo*, vynásobí se posledním operandem (+1 nebo -1) a ten se nahradí součinem.

Když se zjistí *levá závorka* a poslední operand byl -1, přidá se k seznamu operátorů znaménko násobení, aby se později u obsahu v závorce obrátilo znaménko. Byl-li poslední operand +1, odstraní se ze seznamu.

## Příklad s unárními operátory

$$4.5 - -( -7 + + 1 ) = -1.5$$



| Zásobník                | Seznam operandů<br>Seznam operátorů | Zbývající část výrazu | Stav                |
|-------------------------|-------------------------------------|-----------------------|---------------------|
|                         | [ ]<br>[ ]                          | 4.5 - -( -7 + + 1 )   | před operandem      |
|                         | [ 4.5 ]<br>[ ]                      | - -( -7 + + 1 )       | za operandem        |
|                         | [ 4.5, -1 ]<br>[ - ]                | -( -7 + + 1 )         | před operandem      |
|                         | [ 4.5, -1 ]<br>[ -, * ]             | ( -7 + + 1 )          | unární minus        |
| [ 4.5, -1 ]<br>[ -, * ] | [ -1 ]<br>[ ]                       | -7 + + 1 )            | před operandem      |
| [ 4.5, -1 ]<br>[ -, * ] | [ -1 * 7 ]<br>[ ]                   | 7 + + 1 )             | unární minus        |
| [ 4.5, -1 ]<br>[ -, * ] | [ -7 ]<br>[ ]                       | + + 1 )               | za operandem        |
| [ 4.5, -1 ]<br>[ -, * ] | [ -7, +1 ]<br>[ + ]                 | + 1 )                 | před operandem      |
| [ 4.5, -1 ]<br>[ -, * ] | [ -7, +1 * 1 ]<br>[ + ]             | 1 )                   | unární plus         |
| [ 4.5, -1 ]<br>[ -, * ] | [ -7, 1 ]<br>[ + ]                  | )                     | za operandem        |
|                         | [ 4.5, -1, -6 ]<br>[ -, * ]         |                       | konec výrazu        |
|                         | [ 4.5, 6 ]<br>[ - ]                 |                       | po násobení -1 * -6 |
|                         | [ -1.5 ]<br>[ ]                     |                       | po odečítání        |



## Podrobný popis stavového stroje

Stavový stroj začíná stavem PRED\_OPERANDEM a čte první znak. Pak opakuje cyklus, dokud nepřečte poslední znak. Když byl ve stavu

PRED\_OPERANDEM přečten znak

**mezera** (white-space), přečte další znak a zůstane ve stejném stavu

**+** nebo **−**, vyřeší unární operátor: pro **−** přidá k seznamu *operandů* číslo **−1**, pro **+** přidá číslo **1**, přečte další znak a přejde do stavu UNARNI\_OPERATOR

**číslice** nebo **tečka**, převede "číselný" řetězec na číslo pomocí funkce `cislo_exp()`, přidá číslo do seznamu operandů, provede operace *násobení a dělení* v seznamu operandů a přejde do stavu ZA\_OPERANDEM (další znak už nečte)

**písmeno**, provede funkci `identifikator()`, tedy přečte identifikátor, zjišťuje jeho *hodnotu* ze slovníku, *přidá ji k seznamu operandů*, když ji nenajde, hlásí chybu, přejde do stavu ZA\_OPERANDEM

**levá závorka**, přičte jedničku k čítači levých závorek, uloží seznamy do zásobníků, přečte další znak a přejde do stavu PRED\_OPERANDEM (*vstoupí do rekurze*)

**jiný**, ohlásí chybu, přečte další znak a zůstane ve stejném stavu

ZA\_OPERANDEM přečten znak

**mezera** (white-space), přečte další znak a zůstane ve stejném stavu

**operátor**, přidá operátor k seznamu, přečte další znak a přejde do stavu PRED\_OPERANDEM

**pravá závorka**, přičte jedničku k čítači pravých závorek, provede výpočet zbývajících operací v seznamu, obnoví seznamy operandů a operátorů ze zásobníků, *výsledek výpočtu* přidá k seznamu operandů, provede dodatečné násobení a dělení, přečte další znak a zůstává ve stejném stavu

**středník**, přečte další znak (přeskočí středník) a zůstane ve stejném stavu

**jiný**, ohlásí chybu, přečte další znak a zůstane ve stejném stavu

UNARNI\_OPERATOR přečten znak

**mezera** (white-space), přečte další znak a zůstane ve stejném stavu

**+** nebo **−**, vyřeší unární operátor: vybere poslední operand ze seznamu, vynásobí jej číslem **+1** nebo **−1**, a výsledek přidá zpět do zásobníku operandů (na místo vybraného).

**číslice** nebo **tečka**, převede "číselný" řetězec na číslo pomocí funkce `cislo_exp()`, vynásobí číslo posledním operandem ze seznamu (**1** nebo **−1**), a výsledek dosadí na jeho místo, provede přednostní operace *násobení a dělení* a přejde do stavu ZA\_OPERANDEM (další znak už nečte)

**levá závorka**, přičte jedničku k čítači pravých závorek. Je-li poslední operand **−1**, přidá k seznamu operátorů znaménko násobení pro tento operand (aby vyšlo záporné znaménko před levou závorkou). Je-li poslední operand **1**, odstraní jej. Uloží seznamy do zásobníku. Přečte další znak a přejde do stavu PRED\_OPERANDEM.

**jiný**, ohlásí chybu, přečte další znak a zůstane ve stejném stavu

Na konci výrazu provede zbývajících operace v seznamu operandů a získá *výsledek* výrazu. Když cyklus skončí ve stavu PRED\_OPERANDEM, ohlásí program chybu, že za operátorem chybí výraz, a chybu v počtu závorek.

# Programové řešení

Program pro výpočet je zapsán v *modulu* `vypocet_aritmetického_vyrazu.py`. Modul se skládá z *hlavní funkce*

`aritmeticky_vyraz(indata)`

s několika *vnitřními pomocnými funkcemi*. Parametr `indata` je buď otevřený textový soubor, nebo textový řetězec. Hlavní funkce používá další funkce a třídy z modulu `spolecne_funkce.py`, které jsme již poznali [výše](#) u kontroly aritmetického výrazu.

## Globální příkazy pro data

```
import spolecne_funkce as sf

operands = [] # seznam operandů
operators = [] # seznam operátorů
zasobnik_operandu = sf.Zasobnik() # zásobník operandů
zasobnik_operatoru = sf.Zasobnik() # zásobník operátorů
```

## Hlavní výpočetní funkce

```
def aritmeticky_vyraz(indata, log=True):
    """*** Funkce kontroluje správnost aritmetického výrazu
    s celými čísly, se závorkami a vnořováním.

    Parametr indata je buď otevřený vstupní soubor nebo znakový řetězec.

    Parametr log je v této funkci jen formální, nezpůsobuje žádnou činnost.
    Ve volání může chybět, nebo to může být True nebo False.
    """
    global operands, operators, zasobnik_operandu, zasobnik_operatoru

    # ... zde jsou pomocné funkce popsány v dalším odstavci

    PRED_OPERANDEM = 1
    ZA_OPERANDEM = 2
    UNARNI_OPERATOR = 3
    pocet_lz = 0
    pocet_pz = 0
    sf.nastala_chyba = False

    # index sf.ind pro čítání znaků musí být už nastavený
    # první znak sf.znak musí být už přečtený

    stav = PRED_OPERANDEM

    # Začátek cyklu
    while True:
        if sf.konec or sf.znak == ';':
            break

        if stav == PRED_OPERANDEM: # čekám unární operátor, číslici, levou závorku
            if sf.je_mezera(sf.znak):
                sf.dalsi3(indata)
                continue
            elif sf.je_plus_minus(sf.znak): # unární operátor
                if sf.znak == '-':
                    operands.append(-1)
                else:
                    operands.append(1)
                stav = UNARNI_OPERATOR
                sf.dalsi3(indata)
                continue
            elif sf.je_cislice(sf.znak) or sf.znak == '.': # číslice nebo tečka
                cis = sf.cislo_exp(indata)
                vysledek = float(cis)
                operands.append(vysledek)
                nasobeni_deleni() # provedu přednostní operace násobení a dělení
                # další znak už nečtu, byl přečten po sestavení čísla
                stav = ZA_OPERANDEM
                continue
            elif sf.je_pismeno(sf.znak):
                ident = sf.identifikator(indata) # přečtu identifikátor (jméno proměnné)
```

```

hodnota_jmena = sf.slovník.get(ident) # zjistím jeho hodnotu ze slovníku
if hodnota_jmena is not None: # a přidám ji do seznamu operandů
    operandy.append(hodnota_jmena)
    stav = ZA_OPERANDEM
    continue
else:
    sf.chyba(f"nedefinované jméno {ident} ve stavu PRED_OPERANDEM")
    sf.dalsi3(indata)
    continue
elif sf.znak == '(': # levá závorka - vstupuji do rekurze
    pocet_lz += 1
    ulozeni_seznamu_do_zasobniku()
    sf.dalsi3(indata)
    stav = PRED_OPERANDEM # zpracuj výraz rekurzí
    continue
else:
    sf.chyba(f"chybný znak {sf.znak} ve stavu PRED_OPERANDEM")
    sf.dalsi3(indata)
    continue

if stav == ZA_OPERANDEM: # čekám operátor, pravou závorku, středník
    if sf.je_mezera(sf.znak):
        sf.dalsi3(indata)
        continue
    elif sf.je_operator(sf.znak): # operátor
        operatory.append(sf.znak) # zapíšu operátor do seznamu
        stav = PRED_OPERANDEM
        sf.dalsi3(indata)
        continue
    elif sf.znak == ')': # pravá závorka - vystupuji z rekurze
        pocet_pz += 1
        # provedu zbývající operace na seznamech (hlavně sčítání a odčítání)
        vysledek = zbyle_operace()
        # obnovuji seznamy ze zásobníků, nejsou-li prázdné
        if not (zasobnik_operandu.empty() or zasobnik_operatoru.empty()):
            obnova_seznamu_ze_zasobniku()
            operandy.append(vysledek) # přidám výsledek do seznamu operandů
            nasobeni_deleni() # provedu dodatečné násobení a dělení
        sf.dalsi3(indata)
        continue
    elif sf.znak == ';':
        sf.dalsi3(indata) # přeskočím středník
        continue
    else:
        sf.chyba(f"chybný znak {sf.znak} ve stavu ZA_OPERANDEM")
        sf.dalsi3(indata)
        continue

if stav == UNARNI_OPERATOR: # čekám unární operátor, číslo, levou závorku
    if sf.je_mezera(sf.znak):
        sf.dalsi3(indata)
        continue
    if sf.je_plus_minus(sf.znak): # unární operátor
        if sf.znak == '-':
            vysledek = operandy.pop() * -1 # místo operandu dosadím součin s -1
            operandy.append(vysledek)
        else: # plus
            vysledek = operandy.pop() * 1 # místo operandu dosadím součin s +1
            operandy.append(vysledek)
        sf.dalsi3(indata)
        continue
    if sf.je_cislice(sf.znak) or sf.znak == '.':
        cis = sf.cislo_exp(indata)
        vysledek = float(cis)
        vysledek = vysledek * operandy.pop() # krát 1 nebo -1
        operandy.append(vysledek)
        nasobeni_deleni() # provedu přednostní operace, násobení a dělení
        stav = ZA_OPERANDEM
        # další znak už nečtu, byl přečten po sestavení čísla
        continue
    elif sf.znak == '(': # levá závorka
        pocet_lz += 1
        # poslední operand před levou závorkou
        if operandy[-1] == -1: # pochází-li z unárního minus
            operatory.append('*') # přidám znaménko násobení pro operand -1
        elif operandy[-1] == 1: # pochází-li z unárního plus
            operandy.pop() # odstraním operand
        ulozeni_seznamu_do_zasobniku()
        sf.dalsi3(indata)
        stav = PRED_OPERANDEM # zpracuji výraz rekurzí

```

```

        continue
    else:
        sf.chyba(f"chybný znak {sf.znak} ve stavu UNARNI_OPERATOR")
        sf.dalsi3(indata)
        continue
# Konec cyklu
sf.konec = False # zruším příznak konce pro příští volání
# dokončení výrazu
nasobeni_deleni() # provedu přednostní operace, násobení a dělení
vysledek_vyrazu = zbyte_operace()

if stav == PRED_OPERANDEM:
    sf.chyba(f"na konci chybí výraz")
if pocet_pz != pocet_lz:
    sf.chyba(f"počet levých závorek: {pocet_lz}, "
            f"počet pravých závorek: {pocet_pz}")
if sf.nastala_chyba:
    print(f"\nVe výrazu jsou chyby!")
return vysledek_vyrazu

```

## Pomocné vnitřní funkce

```

def operace(levy_operand, operator, pravy_operand):
    vysledek = 0
    if operator == '+':
        vysledek = levy_operand + pravy_operand
    elif operator == '-':
        vysledek = levy_operand - pravy_operand
    elif operator == '*':
        vysledek = levy_operand * pravy_operand
    elif operator == '/':
        if pravy_operand != float(0):
            vysledek = levy_operand / pravy_operand
        else:
            sf.chyba(f"dělení nulou!") # tisknu chybovou zprávu
            vysledek = float(float("+inf")) # dosadím plus nekonečno
    return vysledek

def ulozeni_seznamu_do_zasobniku():
    global operandy, operatory
    zasobnik_operandu.push(operandy)
    zasobnik_operatoru.push(operators)
    operandy = [] # nový prázdný seznam operandů
    operatory = [] # nový prázdný seznam operátorů

def obnova_seznamu_ze_zasobniku():
    global operandy, operatory
    # obnovuji seznamy z vršku zásobníků
    operandy = zasobnik_operandu.pop() # obnovím uložený zásobník operandů
    operatory = zasobnik_operatoru.pop() # obnovím uložený zásobník operátorů

def nasobeni_deleni():
    """
    Funkce zpracuje bezzávorkový výraz: čte operandy a operátory ze seznamů,
    přičemž provede násobení a dělení, protože tyto operace mají přednost
    před sčítáním a odčítáním, takže v seznamech zbudou jen
    sčítací a odčítací operace (jestliže tam nějaké byly).
    """
    if not operandy:
        return
    pravy = operandy.pop() # vyberu pravý operand
    if len(operators) > 0:
        operator = operators.pop() # vyberu operátor
        if sf.je_krat_deleno(operator):
            levy = operandy.pop() # vyberu a odstraním levý operand
            vysledek = operace(levy, operator, pravy)
            operandy.append(vysledek) # dosadím výsledek na místo levého operandu
        else:
            operandy.append(pravy)
            operators.append(operator)
    else: # bez operátoru
        operandy.append(pravy)

def zbyte_operace():
    """

```

```

Funkce zpracuje seznamy zbylé po funkci `nasobeni_deleni()` a provede už jen
sčítací a odčítací operace (jestliže nějaké byly), nebo poslední násobení/dělení
(jestliže byly jen operace násobení a dělení).
"""
ind = 0 # index začátku seznamů
while len(operator) > 0:
    operator = operator[ind] # operátor v zásobníku
    levy = operandy[ind] # levý operand v zásobníku
    while len(operator) > 0:
        if ind+1 < len(operandy): # existuje pravý operand?
            pravy = operandy[ind+1]
            vysledek = operace(levy, operator, pravy)
            # zapíšu výsledek operace na místo levého operandu
            operandy[ind] = vysledek
            levy = vysledek # výsledek se stává levým operandem pro příští výpočet
            del operator[ind] # odstraním operátor
            del operandy[ind + 1] # odstraním pravý operand
            if len(operator) > 0:
                operator = operator[ind] # operátor ze seznamu
        else:
            sf.chyba(f"u operandu {operandy[ind]} a operátoru {operator[ind]} "
                    f"chybí jiný operand")
            return levy # vrátím levý operand
    # bez operátoru
    ind += 1 # zvýším index operandů a operátorů
if len(operandy) > 0:
    operand = operandy.pop()
else:
    operand = None
return operand # vrátím poslední zbývajících operand nebo None

```

## Příklady výpočtu aritmetického výrazu

### Vstupní data v textu

```

text = f"""
4.5 + 1 +
    (7 -
        (10 - 2 * 4)
    )
    / 5 ;"""
print(f"{text} = {aritmeticky_vyraz(text)}")

```

### Tisk výsledku

```

4.5 + 1 +
    (7 -
        (10 - 2 * 4)
    )
    / 5 ; = 6.5

```

### Vstupní data v otevřeném souboru

```

text = "(---1.45 + 61) * (2 + 3) * - 2 + (62. - 2)"
with open("cisla.txt", "w") as outfile:
    outfile.write(text)
infile = open("cisla.txt", "r")
sf.prvni3(infile) # musím dodat první znak výrazu
print(f"{text} = {aritmeticky_vyraz(infile)}")

```

### Tisk výsledku

```

(---1.45 + 61) * (2 + 3) * - 2 + (62. - 2) = -535.5

```

# Výrazy se jmény proměnných a přiřazovací příkazy

K řešení tohoto úkolu definujeme pravidla *přiřazovacího příkazu*. Formální zápis je jednoduchý.

```
příkaz ::= mz identifikátor mz '=' mz výraz mz ';' mz  
příkazy ::= příkaz+
```

Příkaz se skládá ze jména, rovnítka a výrazu a je ukončen středníkem, kde identifikátor a výraz jsou definovány [výše](#). Příkazů můžeme zapsat za sebou více. Identifikátor chápeme jako jméno proměnné. Jména mohou ovšem být i ve výrazu; při výpočtu musí být pochopitelně známa jejich číselná hodnota. Příklad:

```
a = 2;  
b = 3 ;  
c = 1.6 * (2.4 - .4);  
vysledek = 0.23 + (a + b) * c ;
```

Když to spočítáme, vyjde 16.23.

## Podrobný popis stavového stroje

Stavový stroj začíná stavem PRED\_JMENEM a čte první znak. Pak opakuje cyklus, dokud nepřečte poslední znak. Když byl ve stavu

PRED\_JMENEM přečten znak

- písmeno**, provede funkci `identifikator()`, tedy přečte *jméno proměnné* pro levou stranu příkazu a přejde do stavu PRED\_ROVNITKEM

- mezera** (white-space), přečte další znak a zůstane ve stejném stavu

- jiný**, ohlásí chybu, přečte další znak a zůstane ve stejném stavu

PRED\_ROVNITKEM přečten znak

- = rovnítko**, přečte další znak a přejde do stavu PRED\_VYRAZEM

- mezera** (white-space), přečte další znak a zůstane ve stejném stavu

- jiný**, ohlásí chybu, přečte další znak a zůstane ve stejném stavu

PRED\_VYRAZEM přečten znak

- písmeno**, **číslice**, **tečka**, **+**, **-**, **levá závorka**, provede funkci `aritmeticky_vyraz()` a její výsledek zapíše do *slovníku* ke jménu zjištěnému ve stavu PRED\_JMENEM a přejde do stavu ZA\_VYRAZEM

- mezera** (white-space), přečte další znak a zůstane ve stejném stavu

- jiný**, ohlásí chybu, přečte další znak a zůstane ve stejném stavu

ZA\_VYRAZEM přečten znak

- ;** **středník**, přečte další znak a přejde do stavu PRED\_JMENEM (na další přiřazovací příkaz)

- mezera** (white-space), přečte další znak a zůstane ve stejném stavu

- jiný**, ohlásí chybu, přečte další znak a zůstane ve stejném stavu

## Moduly pro výpočet

Program pro výpočet je zapsán v *modulu* `prirazovaci_prikazy.py` a skládá se z jediné funkce

`prirazovaci_prikaz()`.

Funkce přijímá parametr `indata`, který je jako dříve buď otevřený textový soubor, nebo textový řetězec.

Funkce používá další prostředky z *modulu* `spolecne_funkce.py`, které jsme již poznali [výše](#) u kontroly aritmetického výrazu, ale hlavně [výpočetní funkci](#)

`aritmeticky_vyraz()`,

která počítá výraz složený z čísel bez proměnných a je zapsána v *modulu*

`vypocet_aritmetickeho_vyrazu.py`.

## Slovník

Podstatnou částí řešení je tabulka jmen a jejich číselných hodnot vytvořená jako slovník (dictionary), do níž program zapisuje hodnoty číselného výrazu z pravé strany přiřazovacího příkazu. Slovník se také používá ve volané funkci `aritmeticky_vyraz()`, kde z něj program čte hodnoty proměnných.

## Funkce `prirazovaci_prikaz()`

```
import spolecne_funkce as sf
import vypocet_aritmetickeho_vyrazu as va

def prirazovaci_prikaz(indata):

    PRED_JMENEM = 1
    PRED_ROVNITKEM = 2
    PRED_VYRAZEM = 3
    ZA_VYRAZEM = 5
    ident = '' # identifikátor

    sf.nastala_chyba = False # zruším příznak chyby
    sf.ind = 0 # index do vstupního textu potřebný ve funkci dalsi3()
    sf.slovník = dict() # prázdný slovník
    if sf.konec:
        sf.chyba(f"prázdný vstup")
        return
    sf.prvni3(indata) # čtu první znak

    stav = PRED_JMENEM

    while True:
        if sf.konec: # na konci vstupu končím všechny příkazy
            break

        if stav == PRED_JMENEM: # čekám jméno
            if sf.je_pismeno(sf.znak): # písmeno?
                ident = sf.identifikator(indata) # vrátí identifikátor
                stav = PRED_ROVNITKEM
                continue # nečtu další znak
            elif sf.je_mezera(sf.znak):
                sf.dalsi3(indata)
```

```

        continue
    else:
        sf.chyba(f"chybný znak {sf.znak} ve stavu PRED_JMENEM")
        sf.dalsi3(indata)
        continue
if stav == PRED_ROVNITKEM:
    if sf.znak == '=': # rovnítko?
        stav = PRED_VYRAZEM
        sf.dalsi3(indata)
        continue
    elif sf.je_mezera(sf.znak):
        sf.dalsi3(indata)
        continue
    else:
        sf.chyba(f"chybný znak {sf.znak} ve stavu PRED_ROVNITKEM")
        sf.dalsi3(indata)
        continue
if stav == PRED_VYRAZEM:
    if sf.je_cislice(sf.znak) or sf.znak == '.' or sf.je_plus_minus(sf.znak) \
        or sf.znak == '(' or sf.je_pismeno(sf.znak):
        # vypočítám hodnotu aritmetického výrazu
        vysledek = va.aritmeticky_vyraz(indata)
        print(f"{ident} =", vysledek) # tisknu výsledek výrazu
        sf.slovník[ident] = vysledek # zapíšu výsledek ke jménu do slovníku
        stav = ZA_VYRAZEM
        continue
    elif sf.je_mezera(sf.znak):
        sf.dalsi3(indata)
        continue
    else:
        sf.chyba(f"chybný znak {sf.znak} ve stavu PRED_VYRAZEM")
        sf.dalsi3(indata)
        continue
if stav == ZA_VYRAZEM:
    if sf.je_mezera(sf.znak):
        sf.dalsi3(indata)
        continue
    elif sf.znak == ';':
        stav = PRED_JMENEM # za středníkem zahájím nový příkaz
        sf.dalsi3(indata)
        continue
    else:
        sf.chyba(f"chybný sf.znak {sf.znak} ve stavu ZA_VYRAZEM")
        sf.dalsi3(indata)
        continue

print("konec příkazů")
sf.konec = False # zruším globální příznak konce vstupních dat (pro další volání)
print(sf.slovník) # vytisknu adresář s výsledky

```



# Příklady

```
print("\n*Text")

text = f"""a = 2;
b = 3 ;
c = 1.6 * (2.4 - .4);
vysledek = 0.23 + (a + b) * c ;"""
print("-----")
print("Příkazy k výpočtu:")
print(text)
print("-----")
prirazovaci_prikaz(text)

*Text
-----
Příkazy k výpočtu:
a = 2;
b = 3 ;
c = 1.6 * (2.4 - .4);
vysledek = 0.23 + (a + b) * c ;
-----
a = 2.0
b = 3.0
c = 3.2
vysledek = 16.23
konec příkazů
{'a': 2.0, 'b': 3.0, 'c': 3.2, 'vysledek': 16.23}

text = f"""_pole = 531+1;
b=8*4+(1 + 1 + 8*2);
abc = _pole / b +.44;""" # _pole = 532; b = 50; abc = 10.64 + 0.44 = 11.08
print("-----")
print("Příkazy k výpočtu:")
print(text)
print("-----")
prirazovaci_prikaz(text)

-----
Příkazy k výpočtu:
_pole = 531+1;
b=8*4+(1 + 1 + 8*2);
abc = _pole / b +.44;
-----
_pole = 532.0
b = 50.0
abc = 11.08
konec příkazů
{'_pole': 532.0, 'b': 50.0, 'abc': 11.08}

-----
text = f"""a = 1; b = 2;
c = a / b + 0.5;""" # c = 1
print("-----")
print("Příkazy k výpočtu:")
print(text)
print("-----")
prirazovaci_prikaz(text)

-----
Příkazy k výpočtu:
a = 1; b = 2;
c = a / b + 0.5;
-----
a = 1.0
b = 2.0
c = 1.0
konec příkazů
{'a': 1.0, 'b': 2.0, 'c': 1.0}
```

```

print("\n*Soubor")
text = f"""a = (---1.45 + 61) * (2 + 3) * - 2 + (62. - 2);""" # a = 535.5
# zápis zkušebních dat do souboru
# -----
with open("cisla.txt", "w") as outfile:
    outfile.write(text)
# vstup je otevřený soubor
# -----
infile = open("cisla.txt", "r")
print("-----")
print("Příkazy k výpočtu:")
print(text)
print("-----")
prirazovací_prikaz(infile)

*Soubor
-----
Příkazy k výpočtu:
a = (---1.45 + 61) * (2 + 3) * - 2 + (62. - 2);
-----
a = -535.5
konec příkazů
{'a': -535.5}

```